

Task 2:

What is the Gamma Function?

The Gamma function, denoted by the Greek capital letter Γ , is essentially an extension of the factorial function to real and complex numbers. While the standard factorial ($n!$) only applies to non-negative integers, the Gamma function can handle continuous values.

For any positive real number z , the Gamma function is defined by the following integral:

$$\Gamma(z) = \int_{(0 \text{ to } \infty)} [t^{(z-1)} * e^{(-t)}] dt$$

Key Properties:

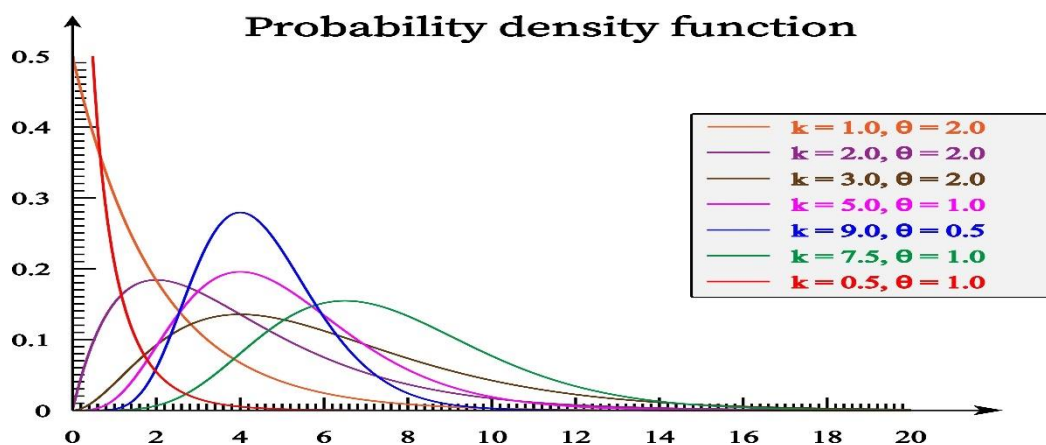
- **Relationship to Factorials:** For any positive integer n , the Gamma function relates to the factorial function as: $\Gamma(n) = (n-1)!$
- **Recursive Property:** For any positive real number z : $\Gamma(z+1) = z * \Gamma(z)$
- **Half-Integer Value:** A commonly used specific value in probability is: $\Gamma(1/2) = \sqrt{\pi}$

Usage of the Gamma Function in Probability

In probability and statistics (and by extension, machine learning), the Gamma function rarely acts as a standalone model. Instead, it serves as a critical mathematical building block—primarily as a **normalizing constant** to ensure that probability density functions (PDFs) integrate to exactly 1.

Here are its most significant applications in probability:

1. The Gamma Distribution



The Gamma function gives its name to the Gamma distribution, a two-parameter family of

continuous probability distributions. It is frequently used to model continuous variables that are always positive and have skewed distributions, such as waiting times until a specific number of events occur (e.g., in queuing theory or reliability engineering).

The probability density function (PDF) using shape parameter α (alpha) and rate parameter β (beta) is written as:

$$f(x) = [\beta^\alpha * x^{(\alpha-1)} * e^{(-\beta x)}] / \Gamma(\alpha)$$

Here, $\Gamma(\alpha)$ is the normalizing constant that ensures the total area under the probability curve equals exactly 1.

2. The Beta Distribution

The Beta distribution models the behavior of random variables limited to intervals of length 1 (typically 0 to 1), making it extremely useful for modeling probabilities or proportions in Bayesian inference.

The Beta distribution relies on the Beta function, $B(\alpha, \beta)$, which is entirely defined by Gamma functions:

$$B(\alpha, \beta) = [\Gamma(\alpha) * \Gamma(\beta)] / \Gamma(\alpha+\beta)$$

3. The Chi-Square Distribution

The Chi-Square distribution is a workhorse in inferential statistics, used heavily for hypothesis testing (like the Chi-Square test for independence) and constructing confidence intervals. It is actually a special case of the Gamma distribution where the shape parameter is $v/2$ (where v represents the degrees of freedom) and the rate parameter is $1/2$. Consequently, its formula relies heavily on $\Gamma(v/2)$.

4. The Dirichlet Distribution (Multivariate Beta)

In machine learning, particularly in natural language processing (like Latent Dirichlet Allocation) and Bayesian statistics, the Dirichlet distribution is used as a prior for categorical and multinomial variables. It is the multivariate generalization of the Beta distribution, and its normalizing constant is a multivariate extension of the Beta function, calculated using a product of Gamma functions.

Task 3:

What is the Hessian Matrix?

When training machine learning models, we usually need to minimize a loss function. To do this, we look at derivatives. While the gradient (the first derivative) tells us the slope or the direction of the steepest descent, the Hessian matrix (the second derivative) gives us information about the curvature of that function.

Mathematically, if we have a function with many variables (like the weights of a neural network), the Hessian is a square matrix that contains all the second-order partial derivatives of that function. It looks like this:

$$H = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \dots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \dots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \dots & \dots & \dots & \dots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \dots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

In short, the gradient tells the model which way is "down," but the Hessian tells the model whether that downward slope is shaped like a bowl, a hill, or a saddle.

Usage of the Hessian Matrix in AI

The Hessian matrix plays a critical role in how machine learning models optimize their parameters and minimize errors. Its main uses include:

1. Faster Optimization (Second-Order Methods)

In standard Gradient Descent, the model only uses the slope to update weights, which means it takes small, sometimes inefficient steps and can bounce around. By using the Hessian matrix (in techniques like Newton's Method), the model understands the exact curvature of the loss landscape. This allows it to take much more direct and accurate steps toward the minimum, speeding up the training process significantly.

2. Identifying Saddle Points

During training, especially in deep learning, the optimizer frequently reaches points where the gradient is exactly zero. Without the Hessian, it is hard to tell if this is a true local minimum (the bottom of the curve) or just a saddle point (where it curves up in one direction but drops off in another). The Hessian helps identify these saddle points so the algorithm knows it hasn't actually reached the bottom and needs to keep optimizing.

3. Practical Approximations (L-BFGS)

In reality, calculating the full Hessian matrix for large models with millions of parameters is computationally too expensive and requires too much memory. To solve this, we use "Quasi-Newton" methods that approximate the Hessian matrix. A common example is the L-BFGS algorithm, which is frequently used under the hood in libraries like Scikit-Learn. It gives the model the benefits of understanding the curvature without running out of computer memory.